MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Window Functions & Pivot - Unpivot

ROW_NUMBER · RANK · DENSE_RANK — CASE WHEN & Conditional Aggregation

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:15 – 1:50	Window Functions Intro & Ranking	**Theory** 📖
1:50 – 2:15	Aggregate & Value Window Functions	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 3:00	Pivot: Rows to Columns (CASE WHEN)	**Theory** 📖
3:00 – 3:20	Unpivot: Columns to Rows (UNION ALL)	**Theory** 📖
3:20 – 3:40	Hands-on: Window Functions & Pivot/Unpivot	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

📖 PART 1 — THEORY

Window Functions

What Are Window Functions?

GROUP BY collapses rows → one row per group

Window functions keep ALL rows → add computed columns

```sql
-- GROUP BY: one row per department
SELECT dept_id, AVG(gpa) FROM student GROUP BY dept_id;
-- Result: 3 rows (one per dept)

-- Window function: every student row + department average
SELECT first_name, gpa, dept_id,
       AVG(gpa) OVER (PARTITION BY dept_id) AS dept_avg
FROM student;
-- Result: ALL student rows with dept average added
```

> 💡 Window functions perform calculations **across a set of rows** related to the current row, **without collapsing** the result.

Window Function Syntax

```
function_name() OVER (
    [PARTITION BY column1, column2, ...]
    [ORDER BY column3 [ASC|DESC], ...]
    [frame_clause] )
```

Clause	Purpose	Optional?
`PARTITION BY`	Divides rows into groups (like GROUP BY but keeps rows)	Yes
`ORDER BY`	Orders rows within each partition	Depends on function
Frame clause	Defines the window of rows for aggregation	Yes

Categories of Window Functions:

1. **Ranking:** ROW_NUMBER, RANK, DENSE_RANK, NTILE
2. **Aggregate**: SUM, AVG, COUNT, MIN, MAX (used with OVER)
3. **Value**: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

ROW_NUMBER()

Assigns a **unique sequential number** to each row within its partition.

```
-- Number students within each department by GPA
SELECT first_name, last_name, dept_id, gpa,
       ROW_NUMBER() OVER (
           PARTITION BY dept_id ORDER BY gpa DESC
       ) AS rank_in_dept
FROM student;
> 💡 No ties — even with equal values, each row gets a unique number.
```

first_name	dept_id	gpa	rank_in_dept
Alice	1	3.9	1
Bob	1	3.7	2
Charlie	1	3.2	3
Diana	2	3.8	1
Eve	2	3.5	2

RANK() vs DENSE_RANK()

```sql
SELECT first_name, gpa,
       ROW_NUMBER() OVER (ORDER BY gpa DESC) AS row_num, -- Always unique
       RANK()       OVER (ORDER BY gpa DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY gpa DESC) AS dense_rank
FROM student;
```

first_name	gpa	row_num	rank	dense_rank
Alice	3.9	1	1	1
Bob	3.8	2	2	2
Charlie	3.8	3	**2**	**2**
Diana	3.5	4	**4**	**3**
Eve	3.2	5	5	4

- **RANK**: Ties get same rank, **gaps** after ties (1, 2, 2, **4**, 5)
- **DENSE_RANK**: Ties get same rank, **no gaps** (1, 2, 2, **3**, 4)

NTILE(n) — Divide into Buckets

Distributes rows into **n** approximately equal groups.

```sql
-- Divide students into 4 quartiles by GPA
SELECT first_name, gpa,
       NTILE(4) OVER (ORDER BY gpa DESC) AS quartile
FROM student;
```

first_name	gpa	quartile
Alice	3.9	1 (top 25%)
Bob	3.8	1
Charlie	3.5	2
Diana	3.2	2
Eve	3.0	3
Frank	2.8	3
Grace	2.5	4 (bottom 25%)

Pivot: Transforming Rows to Columns

What Is Pivoting?

Source data (row-oriented):

student	subject	grade
Alice	Math	A
Alice	Science	B
Bob	Math	B
Bob	Science	A

Pivoted result (column-oriented):

student	Math	Science
Alice	A	B
Bob	B	A

Why Pivot?

> 💡 **Pivot** = rotate rows into columns. Each unique value in one column becomes a new column header.

Common use cases:

1. **Report formatting**: Months as columns (Jan, Feb, Mar...)
2. **Cross-tabulation**: Products × Regions
3. **Scorecards**: Students × Subjects
4. **Dashboards**: Metrics side by side

MySQL has `PIVOT` keyword (in some contexts), but:

> **SQLite does not have a PIVOT keyword** — we use **CASE WHEN** + **GROUP BY** instead. This technique is called **conditional aggregation**.

Pivot with CASE WHEN (Conditional Aggregation)

```sql
-- Source: enrollment with grades per course
-- Goal: one row per student, one column per course

SELECT
    s.first_name,
    MAX(CASE WHEN c.course_name = 'Database Systems'
             THEN e.grade END) AS db_grade,
    MAX(CASE WHEN c.course_name = 'Algorithms'
             THEN e.grade END) AS algo_grade,
    MAX(CASE WHEN c.course_name = 'Calculus'
             THEN e.grade END) AS calc_grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id
GROUP BY s.student_id, s.first_name;
```

How it works:

- `CASE WHEN` checks each row's course name

Step-by-Step: How Conditional Aggregation Works

Step 1: Before GROUP BY (expanded)

student	course	grade	db_grade	algo_grade
Alice	Database	A	**A**	NULL
Alice	Algo	B	NULL	**B**
Bob	Database	B	**B**	NULL
Bob	Algo	A	NULL	**A**

Step 2: After GROUP BY + MAX() // `MAX()` ignores NULLs

student	db_grade	algo_grade
Alice	A	B
Bob	B	A

Pivot: Numeric Aggregation

```sql
-- Monthly revenue pivot
-- Source: orders with order_date and total_amount

SELECT
    strftime('%Y', order_date) AS year,
    ROUND(SUM(CASE WHEN strftime('%m', order_date) = '01'
                   THEN total_amount ELSE 0 END), 2) AS Jan,
    ROUND(SUM(CASE WHEN strftime('%m', order_date) = '02'
                   THEN total_amount ELSE 0 END), 2) AS Feb,
    ROUND(SUM(CASE WHEN strftime('%m', order_date) = '03'
                   THEN total_amount ELSE 0 END), 2) AS Mar,
    ROUND(SUM(CASE WHEN strftime('%m', order_date) = '04'
                   THEN total_amount ELSE 0 END), 2) AS Apr
FROM orders
GROUP BY strftime('%Y', order_date);
```

> 💡 For numeric values, use `SUM()` with `ELSE 0`. For text/grades, use `MAX()`.

Pivot: Count Cross-Tabulation

```sql
-- Cross-tab: How many students per department per GPA bracket

SELECT d.dept_name,
    COUNT(CASE WHEN s.gpa >= 3.5 THEN 1 END) AS 'A (3.5+)',
    COUNT(CASE WHEN s.gpa >= 3.0 AND s.gpa < 3.5 THEN 1 END) AS 'B (3.0-3.4)',
    COUNT(CASE WHEN s.gpa >= 2.0 AND s.gpa < 3.0 THEN 1 END) AS 'C (2.0-2.9)',
    COUNT(CASE WHEN s.gpa < 2.0 THEN 1 END) AS 'F (<2.0)',
    COUNT(*) AS total
FROM student s
JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

dept_name	A (3.5+)	B (3.0-3.4)	C (2.0-2.9)	F (<2.0)	total
CS	3	2	1	0	6
Math	2	3	2	1	8

Unpivot: Transforming Columns to Rows

What Is Unpivoting?

Source data (column-oriented):

student	math_grade	science_grade	english_grade
Alice	A	B	A
Bob	B	A	B

Unpivoted result (row-oriented):

student	subject	grade
Alice	Math	A
Alice	Science	B
Alice	English	A
Bob	Math	B
Bob	Science	A
Bob	English	B

Unpivot with UNION ALL

```sql
CREATE TABLE grade_report (
    student TEXT,
    math TEXT,
    science TEXT,
    english TEXT
);

INSERT INTO grade_report VALUES ('Alice', 'A', 'B', 'A');
INSERT INTO grade_report VALUES ('Bob', 'B', 'A', 'B');

-- Unpivot using UNION ALL
SELECT student, 'Math' AS subject, math AS grade
FROM grade_report
UNION ALL
SELECT student, 'Science', science
FROM grade_report
UNION ALL
SELECT student, 'English', english
FROM grade_report
ORDER BY student, subject;
```

> Each SELECT extracts one column and labels it with a subject name.

Unpivot: Numeric Example

```sql
-- Quarterly sales as columns
CREATE TABLE quarterly_sales (
    product TEXT,
    q1 REAL,
    q2 REAL,
    q3 REAL,
    q4 REAL
);

INSERT INTO quarterly_sales VALUES ('Widget A', 1000, 1500, 1200, 1800);
INSERT INTO quarterly_sales VALUES ('Widget B', 800, 950, 1100, 900);

-- Unpivot to row format
SELECT product, 'Q1' AS quarter, q1 AS revenue FROM quarterly_sales
UNION ALL
SELECT product, 'Q2', q2 FROM quarterly_sales
UNION ALL
SELECT product, 'Q3', q3 FROM quarterly_sales
UNION ALL
SELECT product, 'Q4', q4 FROM quarterly_sales
ORDER BY product, quarter;
```

Pivot vs Unpivot Summary

Pivot	Rows → Columns	`CASE WHEN` + `GROUP BY`
Unpivot	Columns → Rows	`UNION ALL`

When to pivot:

- Reporting: one row per entity, metrics as columns
- Dashboards: side-by-side comparison
- Cross-tabulation: two dimensions

When to unpivot:

- Normalize denormalized data
- Prepare data for charting (most charts want row-format)
- Data cleaning: convert wide tables to long format

☕ BREAK (15 minutes)

💻 PART 4 — CODING PRACTICE

Window Functions & Pivot/Unpivot

Ranking Students

```sql
-- Rank students by GPA (overall)
SELECT first_name, last_name, gpa,
       ROW_NUMBER() OVER (ORDER BY gpa DESC) AS row_num,
       RANK()       OVER (ORDER BY gpa DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY gpa DESC) AS dense_rank
FROM student;

-- Rank students within each department
SELECT first_name, last_name, dept_id, gpa,
       ROW_NUMBER() OVER (
           PARTITION BY dept_id ORDER BY gpa DESC
       ) AS dept_rank
FROM student;

-- Top student per department
WITH ranked AS (
    SELECT first_name, last_name, dept_id, gpa,
           ROW_NUMBER() OVER (
               PARTITION BY dept_id ORDER BY gpa DESC
           ) AS rn
    FROM student
)
SELECT * FROM ranked WHERE rn = 1;
```

NTILE Quartiles

```sql
-- Divide students into GPA quartiles
SELECT first_name, gpa,
       NTILE(4) OVER (ORDER BY gpa DESC) AS quartile,
       CASE NTILE(4) OVER (ORDER BY gpa DESC)
           WHEN 1 THEN 'Top 25%'
           WHEN 2 THEN 'Above Average'
           WHEN 3 THEN 'Below Average'
           WHEN 4 THEN 'Bottom 25%'
       END AS category
FROM student;

-- Grading on a curve: top 10% get A, etc.
SELECT first_name, gpa,
       NTILE(10) OVER (ORDER BY gpa DESC) AS decile,
       CASE
           WHEN NTILE(10) OVER (ORDER BY gpa DESC) = 1 THEN 'A+'
           WHEN NTILE(10) OVER (ORDER BY gpa DESC) <= 3 THEN 'A'
           WHEN NTILE(10) OVER (ORDER BY gpa DESC) <= 5 THEN 'B'
           WHEN NTILE(10) OVER (ORDER BY gpa DESC) <= 7 THEN 'C'
           ELSE 'D'
       END AS curved_grade
FROM student;
```

Aggregate Windows

```sql
-- Each student with department context
SELECT first_name, dept_id, gpa,
       COUNT(*) OVER (PARTITION BY dept_id) AS dept_size,
       ROUND(AVG(gpa) OVER (PARTITION BY dept_id), 2) AS dept_avg,
       MAX(gpa) OVER (PARTITION BY dept_id) AS dept_max,
       MIN(gpa) OVER (PARTITION BY dept_id) AS dept_min,
       ROUND(gpa - AVG(gpa) OVER (PARTITION BY dept_id), 2)
           AS diff_from_avg
FROM student
ORDER BY dept_id, gpa DESC;

-- Percentage of department total
SELECT first_name, dept_id, gpa,
       ROUND(100.0 * gpa / SUM(gpa) OVER (PARTITION BY dept_id), 1)
           AS pct_of_dept_gpa
FROM student
ORDER BY dept_id, gpa DESC;
```

💻 Running Totals

```sql
-- Running total of enrollments per semester
WITH enrollment_counts AS (
    SELECT semester, COUNT(*) AS cnt
    FROM enrollment
    GROUP BY semester
)
SELECT semester, cnt,
       SUM(cnt) OVER (ORDER BY semester) AS running_total
FROM enrollment_counts;

-- Moving average (3-period) on e-commerce orders
SELECT order_id, order_date, total_amount,
       ROUND(AVG(total_amount) OVER (
           ORDER BY order_date
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ), 2) AS moving_avg_3
FROM orders
ORDER BY order_date;
```

💻 LAG/LEAD (cover if time allows)

```sql
-- Compare each order to previous order
SELECT order_id, order_date, total_amount,
       LAG(total_amount) OVER (ORDER BY order_date) AS prev_amount,
       total_amount - LAG(total_amount) OVER (ORDER BY order_date)
           AS change,
       CASE
           WHEN total_amount > LAG(total_amount) OVER (ORDER BY order_date)
               THEN '📈 Up'
           WHEN total_amount < LAG(total_amount) OVER (ORDER BY order_date)
               THEN '📉 Down'
           ELSE '➡️ Same'
       END AS trend
FROM orders
ORDER BY order_date;

-- Days between consecutive orders
SELECT order_id, order_date,
       LAG(order_date) OVER (ORDER BY order_date) AS prev_date,
       JULIANDAY(order_date) -
       JULIANDAY(LAG(order_date) OVER (ORDER BY order_date))
           AS days_between
FROM orders;
```

💻 Window + CTE Combined (cover if time allows)

```sql
-- Top 2 students per department using CTE + window function
WITH ranked_students AS (
    SELECT s.first_name, s.last_name, d.dept_name, s.gpa,
           DENSE_RANK() OVER (
               PARTITION BY s.dept_id ORDER BY s.gpa DESC
           ) AS dept_rank
    FROM student s
    JOIN department d ON s.dept_id = d.dept_id
)
SELECT dept_name, first_name, last_name, gpa, dept_rank
FROM ranked_students
WHERE dept_rank <= 2
ORDER BY dept_name, dept_rank;

-- Cumulative enrollment with date gap filling
WITH RECURSIVE all_semesters AS (
    SELECT MIN(semester) AS sem FROM enrollment
    UNION ALL
    SELECT sem || '_next' FROM all_semesters WHERE LENGTH(sem) < 20
)
SELECT * FROM all_semesters LIMIT 5;
```

Student Grade Pivot

```sql
-- Create sample data
CREATE TABLE IF NOT EXISTS student_grades (
    student_name TEXT,
    subject TEXT,
    grade TEXT,
    score INTEGER
);

DELETE FROM student_grades;
INSERT INTO student_grades VALUES
('Alice', 'Math', 'A', 95),
('Alice', 'Science', 'B', 85),
('Alice', 'English', 'A', 92),
('Bob', 'Math', 'B', 82),
('Bob', 'Science', 'A', 91),
('Bob', 'English', 'C', 74),
('Charlie', 'Math', 'A', 98),
('Charlie', 'Science', 'B', 88),
('Charlie', 'English', 'B', 85);

-- Pivot: one row per student, one column per subject
SELECT student_name,
    MAX(CASE WHEN subject = 'Math' THEN grade END) AS math,
    MAX(CASE WHEN subject = 'Science' THEN grade END) AS science,
    MAX(CASE WHEN subject = 'English' THEN grade END) AS english
FROM student_grades
GROUP BY student_name;
```

Score Pivot with Aggregations

```sql
-- Pivot scores with multiple metrics
SELECT student_name,
    SUM(CASE WHEN subject = 'Math' THEN score ELSE 0 END) AS math_score,
    SUM(CASE WHEN subject = 'Science' THEN score ELSE 0 END) AS science_score,
    SUM(CASE WHEN subject = 'English' THEN score ELSE 0 END) AS english_score,
    SUM(score) AS total_score,
    ROUND(AVG(score), 1) AS avg_score
FROM student_grades
GROUP BY student_name
ORDER BY total_score DESC;

-- Pivot: subject averages
SELECT 'Average' AS metric,
    ROUND(AVG(CASE WHEN subject = 'Math' THEN score END), 1) AS math,
    ROUND(AVG(CASE WHEN subject = 'Science' THEN score END), 1) AS science,
    ROUND(AVG(CASE WHEN subject = 'English' THEN score END), 1) AS english,
    ROUND(AVG(score), 1) AS overall
FROM student_grades;
```

Unpivot — Wide to Long

```sql
-- Create a wide-format table
CREATE TABLE IF NOT EXISTS exam_results (
    student TEXT,
    midterm INTEGER,
    final_exam INTEGER,
    project INTEGER
);

DELETE FROM exam_results;
INSERT INTO exam_results VALUES ('Alice', 85, 92, 88);
INSERT INTO exam_results VALUES ('Bob', 78, 85, 90);
INSERT INTO exam_results VALUES ('Charlie', 92, 88, 95);

-- Unpivot to long format
SELECT student, 'Midterm' AS assessment, midterm AS score
FROM exam_results
UNION ALL
SELECT student, 'Final Exam', final_exam
FROM exam_results
UNION ALL
SELECT student, 'Project', project
FROM exam_results
ORDER BY student, assessment;
```

Unpivot Sales Data

```sql
-- Unpivot quarterly sales to row format
CREATE TABLE IF NOT EXISTS quarterly_rev (
    region TEXT,
    q1 REAL, q2 REAL, q3 REAL, q4 REAL
);

DELETE FROM quarterly_rev;
INSERT INTO quarterly_rev VALUES ('North', 50000, 55000, 60000, 70000);
INSERT INTO quarterly_rev VALUES ('South', 40000, 45000, 42000, 48000);
INSERT INTO quarterly_rev VALUES ('East', 35000, 38000, 40000, 45000);

-- Unpivot
SELECT region, 'Q1' AS quarter, q1 AS revenue FROM quarterly_rev
UNION ALL
SELECT region, 'Q2', q2 FROM quarterly_rev
UNION ALL
SELECT region, 'Q3', q3 FROM quarterly_rev
UNION ALL
SELECT region, 'Q4', q4 FROM quarterly_rev
ORDER BY region, quarter;

-- Unpivoted data is easier to aggregate
WITH unpivoted AS (
    SELECT region, 'Q1' AS quarter, q1 AS revenue FROM quarterly_rev
    UNION ALL
    SELECT region, 'Q2', q2 FROM quarterly_rev
    UNION ALL
    SELECT region, 'Q3', q3 FROM quarterly_rev
    UNION ALL
    SELECT region, 'Q4', q4 FROM quarterly_rev
)
SELECT region, SUM(revenue) AS annual_total,
       ROUND(AVG(revenue), 0) AS quarterly_avg
FROM unpivoted
GROUP BY region
ORDER BY annual_total DESC;
```

University DB Pivot

```sql
-- Enrollment pivot: students × courses
SELECT s.first_name,
    MAX(CASE WHEN c.course_name LIKE '%Database%' THEN e.grade END) AS database_grade,
    MAX(CASE WHEN c.course_name LIKE '%Algorithm%' THEN e.grade END) AS algo_grade,
    MAX(CASE WHEN c.course_name LIKE '%Calculus%' THEN e.grade END) AS calc_grade
FROM student s
LEFT JOIN enrollment e ON s.student_id = e.student_id
LEFT JOIN course c ON e.course_id = c.course_id
GROUP BY s.student_id, s.first_name
ORDER BY s.first_name;

-- Department cross-tab: enrollment count by semester
SELECT d.dept_name,
    COUNT(CASE WHEN e.semester = 'Fall 2024' THEN 1 END) AS fall_2024,
    COUNT(CASE WHEN e.semester = 'Spring 2025' THEN 1 END) AS spring_2025,
    COUNT(*) AS total
FROM department d
JOIN student s ON d.dept_id = s.dept_id
JOIN enrollment e ON s.student_id = e.student_id
GROUP BY d.dept_id, d.dept_name;
```

💻 Pivot + Window Functions

```sql
-- Combine pivot with running totals
WITH monthly AS (
    SELECT strftime('%Y-%m', sale_date) AS month,
           product,
           SUM(amount) AS revenue
    FROM sales
    GROUP BY strftime('%Y-%m', sale_date), product
)
SELECT month, product, revenue,
       SUM(revenue) OVER (
           PARTITION BY product ORDER BY month
       ) AS cumulative_revenue
FROM monthly
ORDER BY product, month;

-- Pivot the cumulative result
WITH monthly AS (
    SELECT strftime('%Y-%m', sale_date) AS month,
           SUM(amount) AS total
    FROM sales
    GROUP BY strftime('%Y-%m', sale_date)
)
SELECT month, total,
       SUM(total) OVER (ORDER BY month) AS running_total
FROM monthly;
```

Summary: Window Functions & Pivot/Unpivot

Operation/Function	Purpose	Type
`ROW_NUMBER()`	Unique sequential number	Window - Ranking
`RANK()`	Rank with gaps on ties	Window - Ranking
`DENSE_RANK()`	Rank without gaps	Window - Ranking
`NTILE(n)`	Divide into n buckets	Window - Ranking
`SUM() OVER()`	Running total	Window - Aggregate
`AVG() OVER()`	Moving average	Window - Aggregate
`LAG() / LEAD()`	Previous/next row value	Window - Value
`CASE WHEN + GROUP BY`	Pivot rows to columns	Transformation
`UNION ALL`	Unpivot columns to rows	Transformation

📝 Practice Exercises (Homework)

1. **Window Functions Exercise:**
   - Rank all students by GPA and show ROW_NUMBER, RANK, DENSE_RANK
   - Use NTILE(5) to create quintiles for GPA distribution
   - Use LAG to find the gap in days between consecutive orders
   - Calculate a 3-order moving average of total_amount
2. **Pivot Exercise:**
   - Create a sales table with columns: `salesperson, product, quarter, revenue`
   - Pivot to show salesperson vs quarter (revenue in cells)
   - Add row totals and column totals
3. **Unpivot #2**

🔮 Next Week Preview

Week 10: SQL Programming

- Variables and assignments
- Control structures: IF/ELSE, CASE, loops
- Error handling
- MySQL vs SQLite programming differences

Preparation:

- 📖 Read about MySQL stored programs
- 💻 Complete pivot/unpivot exercises
- 🤔 Think: How would you automate a monthly report?

Thank You!

Questions?